Chapter 4 | Worked solutions with Python

Self-Attention from Vectors to Matrices

All 15 exercises use the same numbering as the book and the companion exercise sheet. Exercises 1–8 build the interpretation and hand calculations; 9–13 use Python to test shapes and information flow; 14–15 are optional calculus challenges. Work out a prediction before running each check. The explanations are complete without the code; the code makes each claim inspectable.

Unless stated otherwise, use the three-token example with rows *the*, *red*, and *car*, without a mask. Keep full precision until displaying a result. The first mathematical position is Python index 0. A Boolean `True` mask entry means that a key is allowed. These are chosen teaching vectors, not trained representations of the three words.

The notebook is the editable source for both solution PDFs. Open it inside `transformers-book`, install NumPy from `requirements.txt`, and restart the kernel and run all cells in order. The recorded CPU run uses Python 3.12.14 and NumPy 2.5.4. Small floating-point differences across environments are normal.

In [1]

```
from pathlib import Path
import sys
import numpy as np

working_directory = Path.cwd()
for repo_root in (working_directory, *working_directory.parents):
    if (repo_root / "examples/ch04_attention/attention.py").is_file():
        sys.path.insert(0, str(repo_root))
        break
else:
    raise FileNotFoundError("Open this notebook inside transformers-book.")

from examples.ch04_attention.attention import (
    attention_backward, scaled_dot_product_attention, stable_softmax,
)
from examples.ch04_attention import run_examples as demo
from examples.ch04_attention.run_examples import (
    H, W_Q, W_K, W_V, Q, K, V, one_query_example,
)
np.set_printoptions(precision=6, suppress=True)
O, A = scaled_dot_product_attention(Q, K, V)
```

1. Identify the jobs

Problem. For the three-token example, state the shapes and roles of H, W^Q, W^K, W^V, Q, K, V, A, and O. Which objects are learned parameters shared across positions, and which are computed for the current input? Does HW^V combine token positions? Does a row of A give probabilities over the vocabulary?

Start with the jobs, then use the dimensions to check them. There are three token rows, four input features, two matching features, and two value features.

Object	Shape	Job
H	3×4	Input representation at each token
W^Q, W^K	4×2 each	Learned transformations into query/key features
W^V	4×2	Learned transformation into value features
Q, K	3×2 each	Requests and matching information for this input
V	3×2	Contents available to read
A	3×3	One distribution over three key positions per query
O	3×2	One mixture of value features per query

The three W matrices are the learned parameters. Each is shared by all token rows; they remain the same when we run a different sentence through this head. The other arrays are computed for the current input. In particular, training does not store one fixed attention matrix A for every possible sentence.

For one row, $\mathbf{v}_i = \mathbf{h}_i W^V$. Its sum runs over the four features of that row. No other token row enters this projection. In $O = AV$, the sum runs over positions and combines their value vectors. A row of A has three entries because there are three keys, regardless of the vocabulary size. Its weights are therefore not next-word probabilities.

In [2]

```
for name, array in (("H", H), ("W_Q", W_Q), ("W_K", W_K),
                    ("W_V", W_V), ("Q", Q), ("K", K), ("V", V),
                    ("A", A), ("O", O)):
    print(f"{name:3s} {array.shape}")
np.testing.assert_array_equal(H[1] @ W_V, V[1])
assert not np.shares_memory(W_Q, W_K)
```

Output

```
H   (3, 4)
W_Q (4, 2)
W_K (4, 2)
W_V (4, 2)
Q   (3, 2)
K   (3, 2)
V   (3, 2)
A   (3, 3)
O   (3, 2)
```

2. A query with equal scores

Problem. Replace the first query by $(0, 0)$ and leave the keys and values unchanged. Calculate its unmasked weights and output. Explain why a zero score still gives a nonzero weight.

The zero query has dot product zero with every key. Division by $\sqrt{2}$ leaves the row $(0, 0, 0)$. Each exponential is $\exp(0) = 1$, so the denominator is three and the weights are $(1/3, 1/3, 1/3)$.

Position	Value	Weighted contribution
1	$(1, 0)$	$(1/3, 0)$
2	$(0, 2)$	$(0, 2/3)$
3	$(1, 1)$	$(1/3, 1/3)$

Add coordinates separately: the first gives $1/3 + 0 + 1/3 = 2/3$ and the second gives $0 + 2/3 + 1/3 = 1$. The output is $(2/3, 1)$, the ordinary mean of the values. A zero score contributes one to the softmax denominator, not zero. More generally, any constant finite score row produces uniform weights over its allowed keys. The other query rows remain unchanged because each row has its own normalization.

In [3]

```
q2 = Q.copy()
q2[0] = 0
output2, weights2 = scaled_dot_product_attention(q2, K, V)
np.testing.assert_allclose(weights2[0], np.ones(3) / 3)
np.testing.assert_allclose(output2[0], [2 / 3, 1])
np.testing.assert_allclose(output2[1:], O[1:])
print("Weights:", weights2[0])
print("Output:", output2[0])
```

Output

```
Weights: [0.333333 0.333333 0.333333]
Output: [0.666667 1.      ]
```

3. Follow another query

Problem. Use $\mathbf{q}_2 = (0, 1)$ to calculate three dot products, three scaled scores, the weights, and the separate weighted value vectors. Add those vectors and compare with the second row of O. Run `one_query_example(1)` to check each intermediate result. Which sum runs over features, and which sum runs over positions?

Follow the second query, $\mathbf{q}_2 = (0, 1)$, through the unmasked example. Its first coordinate is zero, so only the second coordinate of each key contributes to the dot product. Each query and key has width $d_k = 2$. Divide each dot product by $\sqrt{2}$ before normalizing the row.

Key position	Key	Dot product with $(0, 1)$	Scaled score
1	$(1, 0)$	$0(1) + 1(0) = 0$	0
2	$(0, 1)$	$0(0) + 1(1) = 1$	$1/\sqrt{2}$
3	$(1, 1)$	$0(1) + 1(1) = 1$	$1/\sqrt{2}$

Let $r = \exp(1/\sqrt{2}) \approx 2.028114982$ and $D = 1 + 2r \approx 5.056229963$. The first exponential is $\exp(0) = 1$. The other two exponentials are both r. Softmax divides all three by D, giving the weight row

$$\left(\frac{1}{D}, \frac{r}{D}, \frac{r}{D}\right) \approx (0.197776,\ 0.401112,\ 0.401112).$$

Each value has two coordinates. Multiply both coordinates by the same weight from that key's position.

Position	Value	Exact contribution	Contribution to six decimals
1	$(1, 0)$	$(1/D, 0)$	$(0.197776, 0)$
2	$(0, 2)$	$(0, 2r/D)$	$(0, 0.802224)$
3	$(1, 1)$	$(r/D, r/D)$	$(0.401112, 0.401112)$

Add the first coordinates together and then the second coordinates together. Keeping the weights unrounded gives

$$\mathbf{o}_2 = \left(\frac{1+r}{D}, \frac{3r}{D}\right) \approx (0.598888,\ 1.203336).$$

The second coordinate exceeds one because it is a weighted average of value features $0, 2, 1$. The normalized quantities are the three attention weights.

Python uses zero-based indices, so `one_query_example(1)` selects query 2. The helper returns the scaled scores, weights, individual contributions, and output. Recover the raw dot products by multiplying the scaled scores by `sqrt(2)`. Then compare the row with the second row of the full matrix calculation.

In [4]

```
from examples.ch04_attention.attention import (
    scaled_dot_product_attention,
)
from examples.ch04_attention.run_examples import (
    K, Q, V, one_query_example,
)

scores3, weights3, contributions3, output3 = one_query_example(1)
all_outputs3, all_weights3 = scaled_dot_product_attention(Q, K, V)
raw3 = scores3 * np.sqrt(Q.shape[-1])

exp_score3 = np.exp(1 / np.sqrt(2))
total3 = 1 + 2 * exp_score3
expected_weights3 = np.array([1, exp_score3, exp_score3]) / total3
expected_contributions3 = np.array([
    [1, 0], [0, 2 * exp_score3], [exp_score3, exp_score3],
]) / total3
```

Compare each returned array with the hand calculation. The final two checks compare the output and the weight row with the corresponding rows from the full matrix calculation.

In [5]

```
np.testing.assert_allclose(raw3, [0, 1, 1])
np.testing.assert_allclose(scores3, [0, 1 / np.sqrt(2), 1 / np.sqrt(2)])
np.testing.assert_allclose(weights3, expected_weights3)
np.testing.assert_allclose(contributions3, expected_contributions3)
np.testing.assert_allclose(output3, expected_contributions3.sum(axis=0))
np.testing.assert_allclose(output3, all_outputs3[1])
np.testing.assert_allclose(weights3, all_weights3[1])

with np.printoptions(precision=6, suppress=True):
    print("Raw dot products:", raw3)
    print("Scaled scores:", scores3)
    print("Weights:", weights3)
    print("Contributions, one row per value:")
    print(contributions3)
    print("Output for query 2:", output3)
    print("Second row of the matrix output:", all_outputs3[1])
```

Output

```
Raw dot products: [0. 1. 1.]
Scaled scores: [0.       0.707107 0.707107]
Weights: [0.197776 0.401112 0.401112]
Contributions, one row per value:
[[0.197776 0.      ]
 [0.       0.802224]
 [0.401112 0.401112]]
Output for query 2: [0.598888 1.203336]
Second row of the matrix output: [0.598888 1.203336]
```

A dot product sums over **features**. For a fixed key j, it adds two coordinate products to make one score. Softmax then sums three exponentials over **key positions** for this one query. The read also sums over **positions**, adding one scaled value vector from each position. This last sum removes the position axis and retains the two value coordinates.

4. Change what is read

Problem. Keep Q and K unchanged, but replace the third value by $(3, -1)$. Recalculate the first output. Which intermediate matrices change, and which remain the same? Derive the change $\Delta \mathbf{o}_i$ for any query in terms of A_{i3}. Explain why the attention map alone does not determine the output.

Only the third value changes. Its key remains $(1, 1)$, so no dot product changes. The raw scores, scaled scores, and all of A stay fixed. Let $u = \exp(1/\sqrt{2})$, $a = u/(2u + 1)$, and $b = 1/(2u + 1)$. The first weight row is still (a, b, a). Its new contributions are

$$\mathbf{o}_1' = a(1, 0) + b(0, 2) + a(3, -1) = (4a, 2b - a) \approx (1.604448, -0.005560).$$

The negative second coordinate is valid: it lies between the new value coordinates -1 and 2. We changed the contents of the read, not its weights.

For any query i, all unchanged terms cancel when we subtract the old output:

$$\Delta \mathbf{o}_i = A_{i3}(\mathbf{v}_3' - \mathbf{v}_3) = A_{i3}(2, -2).$$

For query 1 this adds $(2a, -2a)$ to the original $(2a, 2b + a)$, giving the same answer. Every unmasked query changes here because every A_{i3} is positive. The attention map alone cannot determine an output: the same map

can read different contents. Linearity in V holds when Q and K are fixed; changing H in a complete self-attention operation can change all three.

In [6]

```
v4 = V.copy()
v4[2] = [3, -1]
output4, weights4 = scaled_dot_product_attention(Q, K, v4)
np.testing.assert_array_equal(weights4, A)
np.testing.assert_allclose(output4 - O, A[:, 2, None] * [2, -2])
print("First output:", output4[0])
print("Changes in all output rows:")
print(output4 - O)
```

Output

```
First output: [ 1.604448 -0.00556 ]
Changes in all output rows:
[[ 0.802224 -0.802224]
 [ 0.802224 -0.802224]
 [ 1.00698  -1.00698 ]]
```

5. Use two sequence lengths

Problem. Let Q have shape 4×3, K shape 6×3, and V shape 6×5. Write the shapes of the scores, attention weights, and output. Explain why each output still corresponds to a query position. Which dimension determines the scaling divisor?

There are four requests and six available key–value pairs. Each request needs six comparisons, giving a 4×6 grid of scores. The dot products each sum three feature products, so the divisor is $\sqrt{3}$.

Quantity	Shape	Axis meanings
Q	4×3	query, matching feature
$K^\top$	3×6	matching feature, key
$QK^\top$, S, and A	4×6	query, key
V	6×5	key/value position, value feature
$O = AV$	4×5	query, value feature

Scaling and row softmax preserve shape. Every row of A divides its weight among six keys. The product $(4 \times 6)(6 \times 5)$ then combines six values with five features each. For example,

$$O_{4,5} = \sum_{j=1}^{6} A_{4j} V_{j5}.$$

The key-position index disappears into the sum; the query index remains. Four queries therefore produce four output vectors, each of width five. Neither the value width five nor the key count six sets the scale divisor.

In [7]

```
q5, k5, v5 = np.ones((4, 3)), np.ones((6, 3)), np.ones((6, 5))
output5, weights5 = scaled_dot_product_attention(q5, k5, v5)
assert weights5.shape == (4, 6) and output5.shape == (4, 5)
print("Score/weight shape:", weights5.shape)
print("Output shape:", output5.shape)
print("Scaling divisor:", np.sqrt(q5.shape[-1]))
```

Output

```
Score/weight shape: (4, 6)
Output shape: (4, 5)
Scaling divisor: 1.7320508075688772
```

6. Inspect directionality

Problem. Use the unrounded example to compare A_{13} with A_{31}. Explain how symmetric raw scores can produce asymmetric normalized weights.

The raw scores in directions $1 \to 3$ and $3 \to 1$ both equal one. Write $x = 1/\sqrt{2}$ and $u = \exp(x)$. Their scaled scores both equal x, but the competing scores differ:

Query	Scaled scores	Exponentials	Denominator
1	$(x, 0, x)$	$(u, 1, u)$	$2u + 1$
3	$(x, x, 2x)$	(u, u, u^2)	$2u + u^2$

Thus,

$$A_{13} = \frac{u}{2u + 1} \approx 0.401112, \qquad A_{31} = \frac{u}{2u + u^2} \approx 0.248255.$$

For query 1, key 3 ties for the best score. For query 3, key 1 competes with key 3, whose score is higher. Equal pairwise scores do not imply equal shares of two different rows. If $S_{ij} = S_{ji}$, then $A_{ij} = A_{ji}$ exactly when their row denominators agree. That happens for rows 1 and 2 here, but not for rows 1 and 3. Independently learned query and key projections can also make the raw scores asymmetric.

In [8]

```
raw6 = Q @ K.T
np.testing.assert_array_equal(raw6, raw6.T)
assert not np.allclose(A, A.T)
print("A[1,3] in mathematical notation:", A[0, 2])
print("A[3,1] in mathematical notation:", A[2, 0])
print("Ratio:", A[0, 2] / A[2, 0])
```

Output

```
A[1,3] in mathematical notation: 0.4011120926797859
A[3,1] in mathematical notation: 0.2482550782577231
Ratio: 1.6157256298434148
```

7. Separate scaling from shifting

Problem. Start from the raw score row $(1, 0, 1)$. Calculate softmax before scaling, after division by $\sqrt{2}$, and after adding 1000 to every scaled score. Use stable softmax for the last case. Which operations change the weights, their ordering, and the output obtained with the original V? Explain your results using score differences.

Softmax depends on score differences. Dividing by $\sqrt{2}$ reduces the nonzero differences from one to approximately 0.7071. The weights become more even while keeping the same ordering: keys 1 and 3 still tie above key 2. Adding a common constant changes no difference, so it leaves every weight unchanged.

Scores supplied to softmax	Weights	Output with the original V
$(1, 0, 1)$	$(0.422319, 0.155362, 0.422319)$	$(0.844638, 0.733044)$
$(1/\sqrt{2}, 0, 1/\sqrt{2})$	$(0.401112, 0.197776, 0.401112)$	$(0.802224, 0.796664)$
Previous row plus 1000	Same as previous row	Same as previous row

For the shifted row, subtract its maximum first. This gives $(0, -1/\sqrt{2}, 0)$ before exponentiation, exactly the same shifted scores as stable softmax of the unshifted scaled row. Avoid forming $\exp(1000)$. In finite precision the shifted inputs may round very slightly differently; the numerical comparison therefore uses a tolerance.

Multiplying the weights by the same V shows that scaling changes the read in this example. A common shift changes neither the weights nor the read. The order of output coordinates is a separate matter from the ranking of keys.

In [9]

```
raw7 = np.array([1.0, 0.0, 1.0])
scaled7 = raw7 / np.sqrt(2)
rows7 = (raw7, scaled7, scaled7 + 1000)
weights7 = [stable_softmax(row) for row in rows7]
for label, weights in zip(("raw", "scaled", "scaled + 1000"), weights7):
    print(f"{label:14s} weights={weights} output={weights @ V}")
assert not np.allclose(weights7[0], weights7[1])
np.testing.assert_allclose(weights7[1], weights7[2], atol=1e-13, rtol=0)
```

Output

```
raw            weights=[0.422319 0.155362 0.422319] output=[0.844638 0.733044]
scaled         weights=[0.401112 0.197776 0.401112] output=[0.802224 0.796664]
scaled + 1000  weights=[0.401112 0.197776 0.401112] output=[0.802224 0.796664]
```

8. Mask at the wrong stage

Problem. Apply ordinary softmax to the first *scaled* score row, $(1/\sqrt{2}, 0, 1/\sqrt{2})$, then set its future weights to zero without renormalizing. Compare the resulting output with the correct causal output $(1, 0)$. Would setting the future scores to zero before softmax exclude those keys? Explain.

The ordinary first weight row is (a, b, a), where $a \approx 0.401112$ and $b \approx 0.197776$. Erasing its future weights without renormalizing leaves $(a, 0, 0)$, with sum a, not one. Its output is $a(1, 0) = (a, 0) \approx (0.401112, 0)$.

The correct masked row keeps only the first score in the denominator:

$$\frac{(\exp(1/\sqrt{2}), 0, 0)}{\exp(1/\sqrt{2})} = (1, 0, 0).$$

The correct output is therefore the entire first value $(1, 0)$. One available key receives the entire unit weight, independently of its finite score. In exact arithmetic, dividing the erased row by its surviving sum would also recover the answer. Masking scores first computes that normalization directly and avoids relying on surviving weights that might already have underflowed.

Setting future scores to zero also fails. Their exponentials are then one, so those keys still receive positive weight. To exclude a key, use the Boolean mask (implemented by replacing its score with $-\infty$), not the finite score zero. An entirely masked row has no distribution and is rejected by this implementation.

In [10]

```
scores8 = Q[0] @ K.T / np.sqrt(2)
allowed8 = np.array([True, False, False])
wrong8 = stable_softmax(scores8) * allowed8
correct8 = stable_softmax(scores8, allowed8)
zeroed_scores8 = np.where(allowed8, scores8, 0.0)
assert stable_softmax(zeroed_scores8)[1] > 0
np.testing.assert_array_equal(correct8, [1, 0, 0])
print("Erase weights afterward:", wrong8, "sum =", wrong8.sum())
print("Wrong output:", wrong8 @ V)
print("Correct weights and output:", correct8, correct8 @ V)
print("Zeroing future scores still gives:", stable_softmax(zeroed_scores8))
```

Output

```
Erase weights afterward: [0.401112 0.       0.      ] sum = 0.4011120926797859
Wrong output: [0.401112 0.      ]
Correct weights and output: [1. 0. 0.] [1. 0.]
Zeroing future scores still gives: [0.50349  0.248255 0.248255]
```

9. Follow the axes

Problem. For $B = 2$, $h = 4$, $n_q = 3$, $n_k = 5$, $d_k = 8$, and $d_v = 6$, write every shape in the attention calculation. Then deliberately use `k.T` and explain why the result is wrong.

Think of the batch and head axes as labels on a grid of matrices. Choose one example and one head. Its query matrix has three rows, its key matrix has five rows, and each query or key has eight features. A query needs one dot product with each of the five keys. Each dot product sums over eight paired features. Once the dot products are available, normalize over the five key positions and read their six-feature values.

Quantity	Shape	Axis meanings
Q	$(2, 4, 3, 8)$	batch, head, query position, query/key feature
K	$(2, 4, 5, 8)$	batch, head, key position, query/key feature
V	$(2, 4, 5, 6)$	batch, head, key position, value feature
Key transpose within each head	$(2, 4, 8, 5)$	batch, head, query/key feature, key position
Raw scores $QK^\top$	$(2, 4, 3, 5)$	batch, head, query position, key position
Scale $\sqrt{8}$	scalar	same scale at every position
Scaled scores S	$(2, 4, 3, 5)$	batch, head, query position, key position
Softmax row maxima and denominators	$(2, 4, 3, 1)$	one of each per query row
Shifted scores, exponentials, and weights A	$(2, 4, 3, 5)$	one entry per query and key
Output $O = AV$	$(2, 4, 3, 6)$	batch, head, query position, value feature

Within each slice, $(3, 8)(8, 5) \to (3, 5)$ contracts the feature width of eight. The softmax in the example uses `axis=-1` and `keepdims=True` for its reductions. The size-one axis lets NumPy subtract each row's maximum and divide by each row's denominator. The second product $(3, 5)(5, 6) \to (3, 6)$ contracts the five key and value positions. Each of the three query positions retains its own six-coordinate output.

In [11]

```
from examples.ch04_attention.attention import (
    scaled_dot_product_attention,
    stable_softmax,
)
from examples.ch04_attention.run_examples import (
    Q as toy_q, K as toy_k, V as toy_v,
)

rng9 = np.random.default_rng(7)
q9 = rng9.normal(size=(2, 4, 3, 8))
k9 = rng9.normal(size=(2, 4, 5, 8))
v9 = rng9.normal(size=(2, 4, 5, 6))
keys_transposed9 = k9.swapaxes(-2, -1)
raw_scores9 = q9 @ keys_transposed9
scores9 = raw_scores9 / np.sqrt(q9.shape[-1])
row_max9 = scores9.max(axis=-1, keepdims=True)
exponentials9 = np.exp(scores9 - row_max9)
denominator9 = exponentials9.sum(axis=-1, keepdims=True)
output9, weights9 = scaled_dot_product_attention(q9, k9, v9)

assert keys_transposed9.shape == (2, 4, 8, 5)
assert raw_scores9.shape == scores9.shape == (2, 4, 3, 5)
assert row_max9.shape == denominator9.shape == (2, 4, 3, 1)
assert weights9.shape == (2, 4, 3, 5)
assert output9.shape == (2, 4, 3, 6)
np.testing.assert_allclose(weights9, exponentials9 / denominator9)
np.testing.assert_allclose(output9, weights9 @ v9)

slice_weights9 = stable_softmax(q9[1, 2] @ k9[1, 2].T / np.sqrt(8))
np.testing.assert_allclose(weights9[1, 2], slice_weights9)
np.testing.assert_allclose(output9[1, 2], slice_weights9 @ v9[1, 2])
print("One isolated batch/head slice agrees with the batched result.")
print("Weights shape:", weights9.shape)
print("Output shape:", output9.shape)
```

Output

```
One isolated batch/head slice agrees with the batched result.
Weights shape: (2, 4, 3, 5)
Output shape: (2, 4, 3, 6)
```

On a two-dimensional matrix, `.T` swaps the two matrix axes. On `k9`, `.T` reverses **all four** axes. The resulting last two axes are $(4, 2)$, which are the old head and batch axes. They cannot serve as the $(8, 5)$ feature and key axes of the matrix multiplication. The inner dimensions are now 8 and 4, and the leading dimensions $(2, 4)$ and $(8, 5)$ are also incompatible. `swapaxes(-2, -1)` keeps the batch and head labels in place.

In [12]

```
print("Shape with swapaxes:", keys_transposed9.shape)
print("Shape with .T:", k9.T.shape)

try:
    q9 @ k9.T
except ValueError:
    print("ValueError: q9 @ k9.T cannot multiply these shapes.")
else:
    raise AssertionError("Expected the all-axis transpose to fail")
```

Output

```
Shape with swapaxes: (2, 4, 8, 5)
Shape with .T: (8, 5, 4, 2)
ValueError: q9 @ k9.T cannot multiply these shapes.
```

10. Combine padding and causality

Problem. Repeat the toy Q, K, and V across two examples and one head, giving each tensor shape $[2, 1, 3, 2]$. Use valid lengths three and two with *right padding*. Create a Boolean NumPy array `key_valid` with rows `[True, True, True]` and `[True, True, False]`. Keep the toy vectors unchanged, including the vector at the padded position, so the mask's effect is easy to isolate. Combine the shared 3×3 causal mask with `key_valid[:, None, None, :]`, of shape $[2, 1, 1, 3]$. Verify that no query reads the padded key. Inspect the padded query's output and explain why key masking alone need not zero it. Why would also masking out that entire query row trigger an error in our implementation?

Both examples store three queries, three keys, and three values. Example 1 has three real tokens. Example 2 has two real tokens and a right-padding slot at position 3. The vectors in that slot still contain the original toy numbers. Any change therefore comes from the mask.

Let C_{ij} indicate whether key j is at or before query i. Let R_{bj} indicate whether key j is a real token in example b. The allowed mask has entries $C_{ij} \wedge R_{bj}$. Insert singleton head and query axes into `key_valid` to give it shape $(2, 1, 1, 3)$. Boolean AND with the shared $(3, 3)$ lower triangle broadcasts that triangle across examples and repeats each example's key flags across query rows. The combined mask has shape $(2, 1, 3, 3)$, the same as the score array.

In [13]

```
q10, k10, v10 = (
    np.broadcast_to(matrix, (2, 1, 3, 2))
    for matrix in (toy_q, toy_k, toy_v)
)
causal10 = np.tril(np.ones((3, 3), dtype=bool))
key_valid10 = np.array([[True, True, True], [True, True, False]])
key_flags10 = key_valid10[:, None, None, :]
allowed10 = causal10 & key_flags10
outputs10, weights10 = scaled_dot_product_attention(
    q10, k10, v10, allowed_mask=allowed10
)

assert key_flags10.shape == (2, 1, 1, 3)
assert allowed10.shape == weights10.shape == (2, 1, 3, 3)
assert outputs10.shape == (2, 1, 3, 2)
np.testing.assert_array_equal(allowed10[0, 0], causal10)
np.testing.assert_array_equal(weights10[~allowed10], 0.0)
np.testing.assert_array_equal(weights10[1, 0, :, 2], [0, 0, 0])
np.testing.assert_allclose(weights10.sum(axis=-1), 1)
np.testing.assert_allclose(weights10[1, 0, 2], [0.5, 0.5, 0])
np.testing.assert_allclose(outputs10[1, 0, 2], [0.5, 1])

for example in range(2):
    print(f"Example {example + 1} permissions (1 means allowed)")
    print(allowed10[example, 0].astype(int))
    print("Weights")
    print(np.round(weights10[example, 0], 4))
print("Example 2, padded query output:", outputs10[1, 0, 2])
```

Output

```
Example 1 permissions (1 means allowed)
[[1 0 0]
 [1 1 0]
 [1 1 1]]
Weights
[[1.     0.     0.    ]
 [0.3302 0.6698 0.    ]
 [0.2483 0.2483 0.5035]]
Example 2 permissions (1 means allowed)
[[1 0 0]
 [1 1 0]
 [1 1 0]]
Weights
[[1.     0.     0.    ]
 [0.3302 0.6698 0.    ]
 [0.5    0.5    0.    ]]
Example 2, padded query output: [0.5 1. ]
```

Look at the last row of the second example. Its query is $(1, 1)$, so its dot products with the two valid keys $(1, 0)$ and $(0, 1)$ are both 1. The allowed scaled scores tie. Softmax assigns the valid keys weight $1/2$ each and gives the padding key weight zero. The result is

$$\tfrac{1}{2}(1, 0) + \tfrac{1}{2}(0, 2) = (0.5, 1).$$

The third **column** of the mask governs the padding key. The third **row** is still a query over earlier real keys. A separate query-validity policy can discard its output or exclude the corresponding prediction from the loss. If we make the whole row false instead, there is no allowed key and no probability distribution with row sum one. The implementation raises a `ValueError` before doing the normalization.

In [14]

```
empty_query_mask10 = allowed10.copy()
empty_query_mask10[1, 0, 2, :] = False

try:
    scaled_dot_product_attention(q10, k10, v10, empty_query_mask10)
except ValueError as error:
    assert "each query must allow at least one key" in str(error)
    print(f"ValueError: {error}")
else:
    raise AssertionError("Expected rejection of an all-false row")
```

Output

```
ValueError: each query must allow at least one key
```

11. Test the future-information rule

Problem. Copy H and add $(2, -1, 3, -2)$ to its third row only. Recompute Q, K, and V with the original projections. Check that the first two causal output rows are unchanged and equal a run on the original two-position prefix. Compare with unmasked attention. Explain why a causal mask cannot guarantee this property if earlier input rows already contain information from future positions.

The modified third input row is $(3, 0, 4, -1)$. The chosen projections turn it into query and key $(3, 0)$ and value $(4, -1)$. No earlier input row changed, so the first two queries, keys, and values stay the same.

Query 1 can read only position 1. Query 2 can read positions 1 and 2. Neither can read the changed third key or value, so their causal outputs remain $(1, 0)$ and approximately $(0.330238, 1.339523)$. Running the two-position prefix alone gives the same results. In unmasked attention the changed third position is available to both queries, and their outputs change.

This is an intervention on the input, not merely an inspection of zero weights. It checks the whole path from H through the projections and the masked read. The guarantee requires that earlier input rows themselves are unchanged. If an earlier unmasked layer had already mixed the third token into row 1, then row 1 could carry future information even when the current mask allows only its own key. Causality must hold through every path in the decoder.

In [15]

```
h11 = H.copy()
h11[2] += [2, -1, 3, -2]
q11, k11, v11 = h11 @ W_Q, h11 @ W_K, h11 @ W_V
causal11 = np.tril(np.ones((3, 3), dtype=bool))
before11, _ = scaled_dot_product_attention(Q, K, V, causal11)
after11, _ = scaled_dot_product_attention(q11, k11, v11, causal11)
prefix11, _ = scaled_dot_product_attention(
    Q[:2], K[:2], V[:2], causal11[:2, :2]
)
unmasked11, _ = scaled_dot_product_attention(q11, k11, v11)
np.testing.assert_array_equal(after11[:2], before11[:2])
np.testing.assert_allclose(after11[:2], prefix11)
assert not np.allclose(unmasked11[:2], O[:2])
print("Causal prefix, before and after the future change:")
print(after11[:2])
print("Unmasked prefix before:")
print(O[:2])
print("Unmasked prefix after:")
print(unmasked11[:2])
```

Output

```
Causal prefix, before and after the future change:
[[1.       0.      ]
 [0.330238 1.339523]]
Unmasked prefix before:
[[0.802224 0.796664]
 [0.598888 1.203336]]
Unmasked prefix after:
[[ 3.113095 -0.557784]
 [ 1.241275  0.758725]]
```

12. Keep keys and values paired

Problem. Keep Q fixed and reorder the rows of both K and V using the index array `[2, 0, 1]`. Compare the unmasked outputs and explain the result by following the attention columns. Repeat with a causal mask, reordering the original mask's columns with the keys. This represents the original permissions in a new storage order; constructing a new lower triangle would impose different permissions. Finally, reorder only K and explain why the original read is generally lost.

Keep the queries where they are. Python indices `[2, 0, 1]` put the old third key and value first, the old first pair second, and the old second pair third. A score column travels with its key because the dot product depends on that key and the unchanged query. The row's softmax denominator uses the same three scores, so the corresponding weights travel with their columns. Each weight still multiplies the same value.

For example, the original first query adds $A_{11}v_1 + A_{12}v_2 + A_{13}v_3$. The new storage order adds $A_{13}v_3 + A_{11}v_1 + A_{12}v_2$. The summands are the same. The code checks both the rearranged weights and the unchanged outputs. The tolerance allows for small changes in addition order in floating-point arithmetic.

In [16]

```
permutation12 = np.array([2, 0, 1])
keys12, values12 = toy_k[permutation12], toy_v[permutation12]
output12, weights12 = scaled_dot_product_attention(toy_q, toy_k, toy_v)
paired_output12, paired_weights12 = scaled_dot_product_attention(
    toy_q, keys12, values12
)
np.testing.assert_allclose(
    paired_weights12, weights12[:, permutation12], atol=1e-15
)
np.testing.assert_allclose(paired_output12, output12, atol=1e-15)
print("Original first weight row:", np.round(weights12[0], 4))
print("Reordered first weight row:", np.round(paired_weights12[0], 4))
print("Unmasked output, equal in both storage orders")
print(np.round(paired_output12, 4))
```

Output

```
Original first weight row: [0.4011 0.1978 0.4011]
Reordered first weight row: [0.4011 0.4011 0.1978]
Unmasked output, equal in both storage orders
[[0.8022 0.7967]
 [0.5989 1.2033]
 [0.7517 1.    ]]
```

With a causal mask, each allowed flag belongs to a particular query and key pair. Query 1 could originally read only key 1, which now occupies slot 2. Its new permission row must be `[False, True, False]`. Use the original mask's columns in the order `[2, 0, 1]`. The allowed scores and their paired values stay together, so the same weighted sum is computed.

In [17]

```
causal12 = np.tril(np.ones((3, 3), dtype=bool))
causal_output12, causal_weights12 = scaled_dot_product_attention(
    toy_q, toy_k, toy_v, causal12
)
moved_mask12 = causal12[:, permutation12]
moved_output12, moved_weights12 = scaled_dot_product_attention(
    toy_q, keys12, values12, moved_mask12
)
np.testing.assert_allclose(
    moved_weights12, causal_weights12[:, permutation12], atol=1e-15
)
np.testing.assert_allclose(moved_output12, causal_output12, atol=1e-15)

print("Reordered causal permissions (1 means allowed)")
print(moved_mask12.astype(int))
print("Causal output, equal in both storage orders")
print(np.round(moved_output12, 4))
```

Output

```
Reordered causal permissions (1 means allowed)
[[0 1 0]
 [0 1 1]
 [1 1 1]]
Causal output, equal in both storage orders
[[1.     0.    ]
 [0.3302 1.3395]
 [0.7517 1.    ]]
```

A fresh lower triangle uses the **new slot numbers** to decide what is available. In the first row it allows slot 1, which stores the old third value $(1, 1)$. The original causal read returns the old first value $(1, 0)$.

Reordering only the keys changes which values the weights multiply even without a mask. For the first query, the new weights are $[A_{13}, A_{11}, A_{12}]$. With values left in their old order, the result is $A_{13}v_1 + A_{11}v_2 + A_{12}v_3$. The second value now receives the old first key's weight. The notebook's numerical check uses the same toy inputs in both counterexamples.

In [18]

```
wrong_causal12, _ = scaled_dot_product_attention(
    toy_q, keys12, values12, causal12
)
keys_only12, keys_only_weights12 = scaled_dot_product_attention(
    toy_q, keys12, toy_v
)

np.testing.assert_allclose(wrong_causal12[0], toy_v[2])
assert not np.allclose(wrong_causal12[0], causal_output12[0])
np.testing.assert_allclose(keys_only_weights12, paired_weights12)
assert not np.allclose(keys_only12[0], output12[0])

print("First causal output with original permissions:", causal_output12[0])
print("First causal output with a fresh triangle:", wrong_causal12[0])
print("First unmasked output:", np.round(output12[0], 4))
print("First unmasked output after moving only K:",
      np.round(keys_only12[0], 4))
```

Output

```
First causal output with original permissions: [1. 0.]
First causal output with a fresh triangle: [1. 1.]
First unmasked output: [0.8022 0.7967]
First unmasked output after moving only K: [0.5989 1.    ]
```

13. Separate order from parallel computation

Problem. Without a mask or position information, permute all input rows using the same index array and check that the output rows move with them. Explain why the operation does not assign an absolute position to a token. Then explain why all queries within one layer can be evaluated together, even though the next layer and the next generation step still depend on earlier computation.

Start from the three input rows H and place them in order `[2, 0, 1]` **before** forming queries, keys, and values. Each projection applies the same matrix to every row, so the new Q, K, and V all inherit the same permutation. No positional feature is added and all query–key pairs are allowed.

As in Exercise 12, moving the key and value pairs keeps a query's weighted sum unchanged. This time the query rows move too, so the output for the old third query appears first. If $\pi(i)$ maps a new slot to its old index, the scores at new slots (i, j) are the old scores at $(\pi(i), \pi(j))$. Softmax moves the weights in the same way, and the value sum gives the old output at $\pi(i)$. The new weight matrix is the old matrix indexed by the permutation on both axes. The new output is the old output indexed only on its query axis.

In [19]

```
from examples.ch04_attention.run_examples import H, W_Q, W_K, W_V

permutation13 = np.array([2, 0, 1])
h_permuted13 = H[permutation13]
q13 = h_permuted13 @ W_Q
k13 = h_permuted13 @ W_K
v13 = h_permuted13 @ W_V
output13, weights13 = scaled_dot_product_attention(toy_q, toy_k, toy_v)
new_output13, new_weights13 = scaled_dot_product_attention(q13, k13, v13)

np.testing.assert_allclose(q13, toy_q[permutation13])
np.testing.assert_allclose(k13, toy_k[permutation13])
np.testing.assert_allclose(v13, toy_v[permutation13])
np.testing.assert_allclose(
    new_weights13, weights13[np.ix_(permutation13, permutation13)]
)
np.testing.assert_allclose(new_output13, output13[permutation13])

print("Original outputs, in old row order")
print(np.round(output13, 4))
print("New outputs, in old row order [3, 1, 2]")
print(np.round(new_output13, 4))
```

Output

```
Original outputs, in old row order
[[0.8022 0.7967]
 [0.5989 1.2033]
 [0.7517 1.    ]]
New outputs, in old row order [3, 1, 2]
[[0.7517 1.    ]
 [0.8022 0.7967]
 [0.5989 1.2033]]
```

This property is called **permutation equivariance**. Moving an input to a different row moves its output with it. Under these conditions, the calculation uses a token's projected features and the collection of projected keys and values. The row index organizes storage and contributes no absolute-position signal. Adding positional information or fixing a causal mask to the chronological order changes the conditions of this check.

There is a separate question about which calculations depend on each other. Within one attention layer, once its input rows are available, all of that layer's queries, keys, and values can be projected together. Each query's output reads those same input projections. It does not use any other query's output from that layer. Matrix products and row-wise softmax can therefore evaluate the query rows together. This still holds for a known sequence with a causal mask because the mask supplies each row's permissions before the calculation.

The next layer takes the preceding layer's outputs as input, so that dependency is between layers. Autoregressive generation has an additional dependency between time steps. The next token selected at the current step becomes part of the prefix used at the following step. The new token's input is available only after that selection.

14. Choose a finite-difference step (advanced)

Problem. Use the demonstration's `gradient_max_error` helper with its optional `step` argument. Use the toy Q, K, and V, the causal mask, and incoming gradient with rows $(0.2, -0.4)$, $(0.5, 0.3)$, and $(-0.1, 0.6)$. Compare maximum absolute errors for $\varepsilon \in \{10^{-2}, 10^{-4}, 10^{-6}, 10^{-8}\}$, holding the inputs, mask, and incoming gradient fixed. Why can a step that is too large or too small make the estimate less accurate?

Imagine increasing a single entry of the query matrix Q by a tiny amount. The change propagates through dot products, softmax, and the weighted value sum. We want to check the derivative of one scalar loss with respect to that entry. The same procedure works for entries of the key and value matrices K and V.

Let O be the attention output and G the demonstration's fixed incoming gradient. Use the scalar loss

$$L = \langle O, G \rangle = \sum_{i,j} O_{ij} G_{ij}.$$

This choice gives $\partial L / \partial O = G$. To probe a coordinate θ in Q, K, or V, hold everything else fixed and evaluate the same loss twice. Its central-difference derivative is

$$D_\varepsilon L(\theta) = \frac{L(\theta + \varepsilon) - L(\theta - \varepsilon)}{2\varepsilon}.$$

The mask stays fixed in both evaluations. We check every coordinate of the three independent inputs and take the maximum absolute difference from the manual backward pass.

In [20]

```
from examples.ch04_attention import run_examples as demo
from examples.ch04_attention.attention import (
    attention_backward,
    scaled_dot_product_attention,
)

causal = np.tril(np.ones((3, 3), dtype=bool))
incoming = np.array(
    [[0.2, -0.4], [0.5, 0.3], [-0.1, 0.6]], dtype=np.float64
)
qkv = (demo.Q, demo.K, demo.V)
assert all(x.dtype == np.float64 for x in qkv)
print("Fixed causal mask, 1 means allowed:\n", causal.astype(int))
print("Fixed incoming gradient G:\n", incoming)
```

Output

```
Fixed causal mask, 1 means allowed:
 [[1 0 0]
 [1 1 0]
 [1 1 1]]
Fixed incoming gradient G:
 [[ 0.2 -0.4]
 [ 0.5  0.3]
 [-0.1  0.6]]
```

The canonical helper already accepts `step=1e-6`. Calling it with a different positive finite step changes the size of the two perturbations. Copies protect the original matrices, and resetting each coordinate prevents one probe from affecting the next. We import this helper so that the exercise checks the same code used by the demonstration.

The helper contracts the output difference with G before dividing by 2ε. Because G is fixed, $\langle O_+ - O_-, G \rangle = L_+ - L_-$. This is the same central difference in exact arithmetic.

In [21]

```
steps = (1e-2, 1e-4, 1e-6, 1e-8)
print("     step   maximum absolute error")
for step in steps:
    error = demo.gradient_max_error(*qkv, incoming, causal, step=step)
    print(f"{step:9.0e}   {error:.3e}")
assert demo.gradient_max_error(*qkv, incoming, causal) < 1e-8
```

Output

```
     step   maximum absolute error
    1e-02   1.609e-07
    1e-04   1.595e-11
    1e-06   5.682e-11
    1e-08   7.228e-09
```

At 10^{-2} the error is about 1.61×10^{-7}. Reducing the step to 10^{-4} brings it down to about 1.60×10^{-11}. In exact arithmetic, the two Taylor expansions cancel their even-order terms when subtracted. After dividing by 2ε, the first remaining approximation error is proportional to ε^2 for a smooth function. A large step therefore measures the slope across too wide an interval.

At 10^{-6} the measured error grows slightly, and at 10^{-8} it reaches about 7.23×10^{-9}. The two forward evaluations become almost equal as the step shrinks. Subtracting them discards many shared significant digits, and dividing by a tiny 2ε amplifies the remaining rounding error. The smallest observed error in this float64 sweep comes at 10^{-4}. The balance depends on the scale and curvature of the function and on arithmetic precision.

15. Learn through the projections (advanced)

Problem. Extend the derivatives through $Q = HW^Q$, $K = HW^K$, and $V = HW^V$. Reuse the toy H and projections, but set `W_K[0, 0] = 0.5`, so the query and key weights have different numerical entries. Use the causal mask and an incoming gradient G with rows $(0.2, -0.4)$, $(0.5, 0.3)$, and $(-0.1, 0.6)$, as in the demonstration. Derive gradients for H, W^Q, W^K, and W^V, adding all three paths into H, and check each with central differences. As a starting point, differentiating $Q = HW^Q$ gives $\partial L/\partial W^Q = H^\top(\partial L/\partial Q)$. Treat W^Q and W^K as separate variables throughout.

In Exercise 14 we could move an entry of Q without moving K or V. Now each one is produced from the same three input rows in H. Changing an entry of H can change all three projections, so the derivative with respect to that entry must collect three contributions. Changing a weight in one projection changes only that projection.

Make a copy of the teaching key projection weight matrix W^K and set its first entry to 0.5. Keep the query and key projection weights as separate variables, including during the numerical checks. The two weight matrices have the same shape. Each acts in its own projection, and their values now differ. Keep the causal mask and the incoming gradient G from Exercise 14. The scalar loss is still $\langle O, G\rangle$.

In [22]

```
hidden = demo.H.copy()
w_query = demo.W_Q.copy()
w_key = demo.W_K.copy()
w_value = demo.W_V.copy()
w_key[0, 0] = 0.5

q_proj = hidden @ w_query
k_proj = hidden @ w_key
v_proj = hidden @ w_value
projected_output, projected_weights = scaled_dot_product_attention(
    q_proj, k_proj, v_proj, causal
)
with np.printoptions(precision=6, suppress=True):
    print("Revised K:\n", k_proj)
    print("Causal attention weights:\n", projected_weights)
    print("Output O:\n", projected_output)
print(f"Scalar loss = {np.sum(projected_output * incoming):.12f}")
```

Output

```
Revised K:
 [[0.5 0. ]
 [0.  1. ]
 [0.5 1. ]]
Causal attention weights:
 [[1.       0.       0.      ]
 [0.330238 0.669762 0.      ]
 [0.224606 0.319866 0.455527]]
Output O:
 [[1.       0.      ]
 [0.330238 1.339523]
 [0.680134 1.09526 ]]
Scalar loss = 1.356118664858
```

Write $\bar{X}$ for the derivative $\partial L/\partial X$, an array with the same shape as X. First use the manual attention backward pass to obtain $\bar{Q}$, $\bar{K}$, and $\bar{V}$. These are the three incoming gradients for the projections.

To derive the query projection's gradients, look at coordinate (i, b) of its output. The row index i selects a token, a runs over the four input features, and b selects one of the two query features.

$$Q_{ib} = \sum_a H_{ia} W^Q_{ab}.$$

Changing weight W^Q_{ab} changes Q_{ib} by H_{ia} times the weight change, for every token i. Multiply each contribution by $\bar{Q}_{ib}$ and add across tokens. Changing input feature H_{ia} changes every query coordinate Q_{ib} of that token by W^Q_{ab} times the input change. Multiply by $\bar{Q}_{ib}$ and add across query features. The two sums give

$$\bar{W}^Q_{ab} = \sum_i H_{ia} \bar{Q}_{ib}, \quad \bar{W}^Q = H^\mathsf{T} \bar{Q},$$

$$(\bar{H}_Q)_{ia} = \sum_b \bar{Q}_{ib} W^Q_{ab}, \quad \bar{H}_Q = \bar{Q} (W^Q)^\mathsf{T}.$$

The index being summed tells you which axis the matrix product contracts. The weight derivative sums over token rows, whereas the input derivative sums over the output features of that projection.

The same product rule applies to $K = HW^K$ and $V = HW^V$. The projection weight gradients are

$$\bar{W}^K = H^\mathsf{T} \bar{K}, \qquad \bar{W}^V = H^\mathsf{T} \bar{V}.$$

A perturbation to H enters the loss through the queries, the keys, and the values. The first-order changes along these paths add, giving

$$\bar{H} = \bar{Q}(W^Q)^\mathsf{T} + \bar{K}(W^K)^\mathsf{T} + \bar{V}(W^V)^\mathsf{T}.$$

Each weight gradient is a product of a $(4, 3)$ matrix and a $(3, 2)$ matrix, so it has shape $(4, 2)$. Each term for $\bar{H}$ is a $(3, 2)$ matrix times a $(2, 4)$ matrix, so the sum has shape $(3, 4)$. Those shapes agree with the shapes of the four inputs.

In [23]

```
grad_q, grad_k, grad_v = attention_backward(
    q_proj, k_proj, v_proj, incoming, causal
)
grad_w_query = hidden.T @ grad_q
grad_w_key = hidden.T @ grad_k
grad_w_value = hidden.T @ grad_v

through_query = grad_q @ w_query.T
through_key = grad_k @ w_key.T
through_value = grad_v @ w_value.T
grad_hidden = through_query + through_key + through_value

analytic = (grad_hidden, grad_w_query, grad_w_key, grad_w_value)
parameters = (hidden, w_query, w_key, w_value)
names = ("H", "W_Q", "W_K", "W_V")
with np.printoptions(precision=8, suppress=True):
    for name, value, gradient in zip(names, parameters, analytic):
        assert gradient.shape == value.shape
        print(f"dL/d{name} {gradient.shape}\n{gradient}\n")
```

Output

```
dL/dH (3, 4)
[[-0.05472504 -0.12508993  0.34265859 -0.16616466]
 [ 0.0612618   0.16944318  0.30289416  0.39284816]
 [-0.08343843  0.08073669 -0.04555275  0.27331649]]

dL/dW_Q (4, 2)
[[-0.06908173  0.10945007]
 [-0.07690166  0.12508993]
 [-0.06908173  0.10945007]
 [-0.08472159  0.14072979]]

dL/dW_K (4, 2)
[[-0.13816346 -0.15380332]
 [ 0.10945007  0.12508993]
 [-0.13816346 -0.15380332]
 [ 0.24761354  0.27889326]]

dL/dW_V (4, 2)
[[0.29710584 0.10715184]
 [0.25734141 0.66616466]
 [0.29710584 0.10715184]
 [0.56023557 1.05901282]]
```

For example, consider the first feature of the first input token. The first query can read only the first key under the causal mask. Its one attention weight is always one, so its query gradient is zero. The first key can affect later queries, and its first gradient coordinate is about -0.10945007. The first input feature reaches that key coordinate through the revised weight 0.5. It has no direct path into the value projection because that row of W^V is zero. This gives $\bar{H}_{11} = 0 + 0.5(-0.10945007) + 0 \approx -0.05472504$, the upper-left entry of the notebook's printed gradient.

To check the derivation, rerun the full forward calculation after changing one coordinate of one of the four independent arrays. The numerical helper uses only the scalar loss function, so it can catch a mistake in the analytic projection derivatives. In particular, changing H recomputes all three projections, while changing W^Q leaves W^K fixed and vice versa.

In [24]

```
def projection_loss(h, w_q, w_k, w_v):
    output, _ = scaled_dot_product_attention(
        h @ w_q, h @ w_k, h @ w_v, causal
    )
    return np.sum(output * incoming)


def numerical_projection_gradients(parameters, step=1e-6):
    # Perturb each input entry and rerun the whole forward pass.
    copied = [parameter.copy() for parameter in parameters]
    gradients = []
    for array in copied:
        numerical = np.empty_like(array)
        for index in np.ndindex(array.shape):
            original = array[index]
            array[index] = original + step
            loss_plus = projection_loss(*copied)
            array[index] = original - step
            loss_minus = projection_loss(*copied)
            array[index] = original
            numerical[index] = (loss_plus - loss_minus) / (2 * step)
        gradients.append(numerical)
    return gradients
```

In [25]

```
numerical = numerical_projection_gradients(parameters)
print("Input   maximum absolute error")
for name, exact, estimate in zip(names, analytic, numerical):
    max_error = np.max(np.abs(exact - estimate))
    print(f"{name:<5}   {max_error:.3e}")
    np.testing.assert_allclose(exact, estimate, atol=1e-8, rtol=0)

assert w_query[0, 0] == 1.0 and w_key[0, 0] == 0.5
assert not np.shares_memory(w_query, w_key)
```

Output

```
Input   maximum absolute error
H       1.752e-10
W_Q     1.287e-10
W_K     1.368e-10
W_V     1.056e-10
```

All four comparisons pass the 10^{-8} absolute tolerance in the saved float64 run. The key and query weight gradients differ. Each comes from its own projection, and the numerical test perturbs just one of the two matrices at a time. The match for $\bar{H}$ checks the sum of the three paths against a single perturbation that reaches all of them at once.